ALMATY TECHNOLOGICAL UNIVERSITY

ОТЧЕТ

Дисциплина: Визуальное программирование

Тема: Разработка графического приложения «Employee Management System» (Вариант №17)

Технологический стек: Java 21, JavaFX, FXML, Scene Builder, Maven

Выполнил: Абдыкапаров Даниял ИС24-22

Алматы, 2026

СОДЕРЖАНИЕ

1. Введение и цель проекта
2. Технические требования и техническое задание
3. Архитектура и структура проекта
4. Разработка пользовательского интерфейса (FXML / Scene Builder)
5. Логика приложения и обработка событий (Java Controller)
6. Валидация данных и обработка ошибок
7. Тестирование и пользовательские сценарии
8. Инструкция по запуску и развертыванию
9. Заключение

1. ВВЕДЕНИЕ И ЦЕЛЬ ПРОЕКТА

1.1. Введение

Разработка настольных приложений с графическим пользовательским интерфейсом (GUI) является важным аспектом современного программного обеспечения. В рамках дисциплины «Визуальное программирование» изучены фундаментальные концепции построения GUI на платформе JavaFX с применением декларативной верстки FXML, шаблона проектирования MVC (Model-View-Controller) и инструмента визуального проектирования Scene Builder.

1.2. Цель работы

Закрепление практических навыков проектирования GUI, связывания компонентов управления (Controls), обработки событий пользователя,

валидации пользовательского ввода и реализации фильтрации данных на примере разработки настольного приложения **«Employee Management System» (Система управления сотрудниками, Вариант 17)**.

2. ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ И ТЕХНИЧЕСКОЕ ЗАДАНИЕ

В соответствии с методическими указаниями к СРО №1, разработанное приложение отвечает всем обязательным техническим критериям:

1. **Разделение логики и представления:** Интерфейс спроектирован с использованием декларативного языка FXML, а бизнес-логика и обработка событий вынесены в контроллеры.
2. **Компоновка:** Использовано более двух layout-контейнеров (VBox, HBox, GridPane).
3. **Элементы управления (Controls):** Внедрены разнообразные графические компоненты: TableView, TableColumn, TextField, Label, Button, MenuBar, Menu, MenuItem, ToolBar, Alert.
4. **Отображение данных:** Используется элемент TableView с набором из 10 стартовых демонстрационных записей.
5. **Функциональность:** Реализованы добавление новых записей, редактирование существующих, удаление элементов, а также интерактивная фильтрация/поиск в реальном времени.
6. **Многооконность:** Приложение состоит из двух окон — главного окна системы и модального диалогового окна формы создания/редактирования сотрудника.
7. **Безопасность и валидация:** Реализована проверка всех полей ввода с выводом информационных диалогов Alert без аварийного завершения (crash/exception).

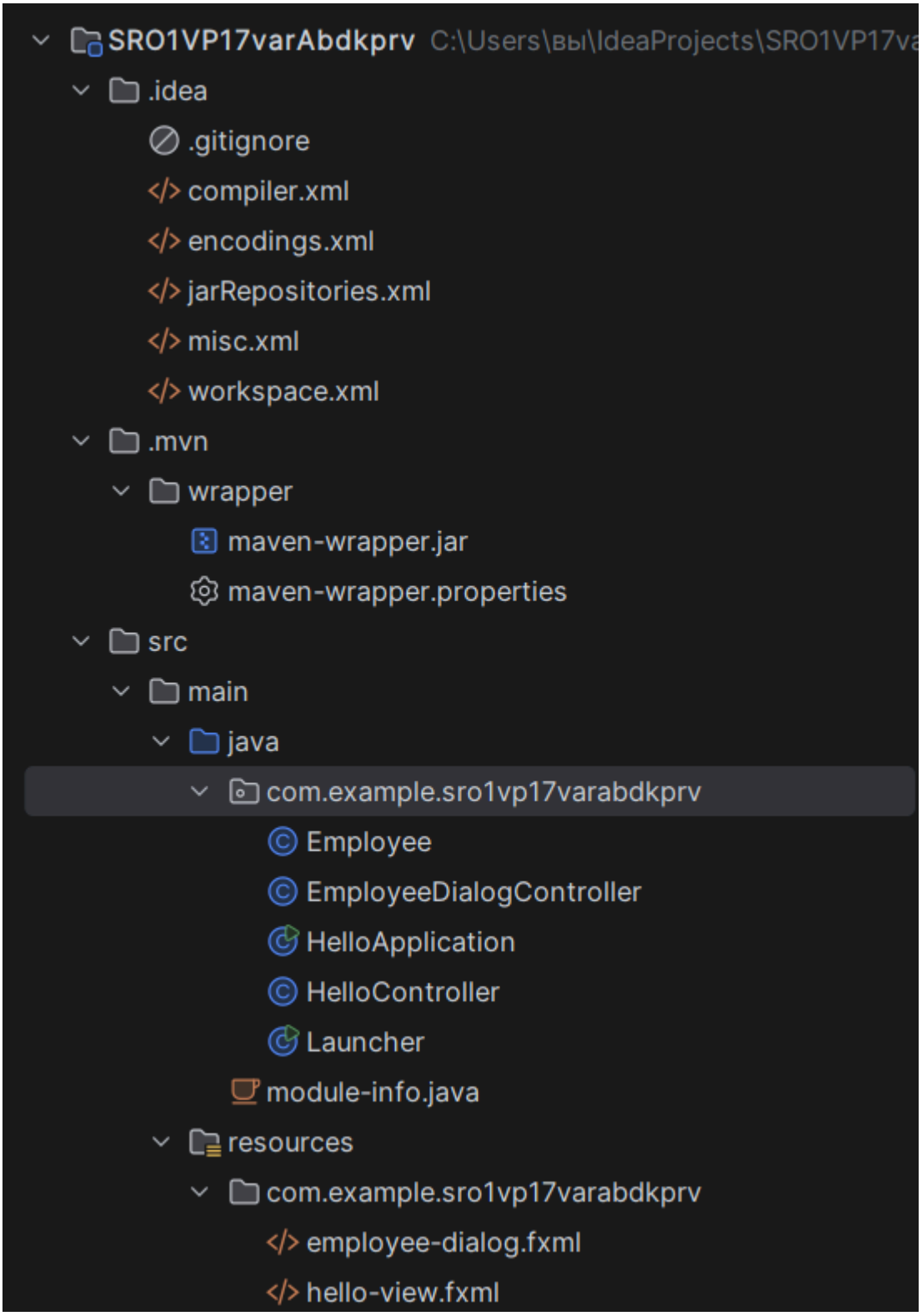
3. АРХИТЕКТУРА И СТРУКТУРА ПРОЕКТА

Проект организован по стандартной структуре Maven-проекта JavaFX с соблюдением принципа разделения ответственности (MVC).

3.1. Дерево файлов проекта

```
SRO1VP17varAbdkprv/
├── pom.xml
└── src/
    └── main/
        ├── java/
        │   └── com/example/sro1vp17varabdkprv/
        │       ├── Employee.java              // Модель данных (Model)
```

```
        │       ├── EmployeeDialogController.java// Контроллер формы ввода
(Controller)
        │       ├── HelloApplication.java       // Главный класс JavaFX (App Entry)
        │       ├── HelloController.java        // Контроллер главного окна (Controller)
        │       ├── Launcher.java               // Точка входа для запуска JVM
        │       └── module-info.java            // Конфигурация модулей Java
        └── resources/
            └── com/example/sro1vp17varabdkprv/
                ├── employee-dialog.fxml        // Разметка диалогового окна (View)
                └── hello-view.fxml             // Разметка главного окна (View)
```



3.2. Описание компонентов MVC

- **Model (Employee.java):** Класс-сущность, хранящий информацию о сотруднике (ФИО, отдел, должность, стаж).
- **View (hello-view.fxml, employee-dialog.fxml):** Декларативное описание пользовательского интерфейса в формате XML.

- **Controller (HelloController.java, EmployeeDialogController.java):** Классы, обрабатывающие действия пользователя (нажатия кнопок, ввод текста) и управляющие состоянием таблицы и полей.

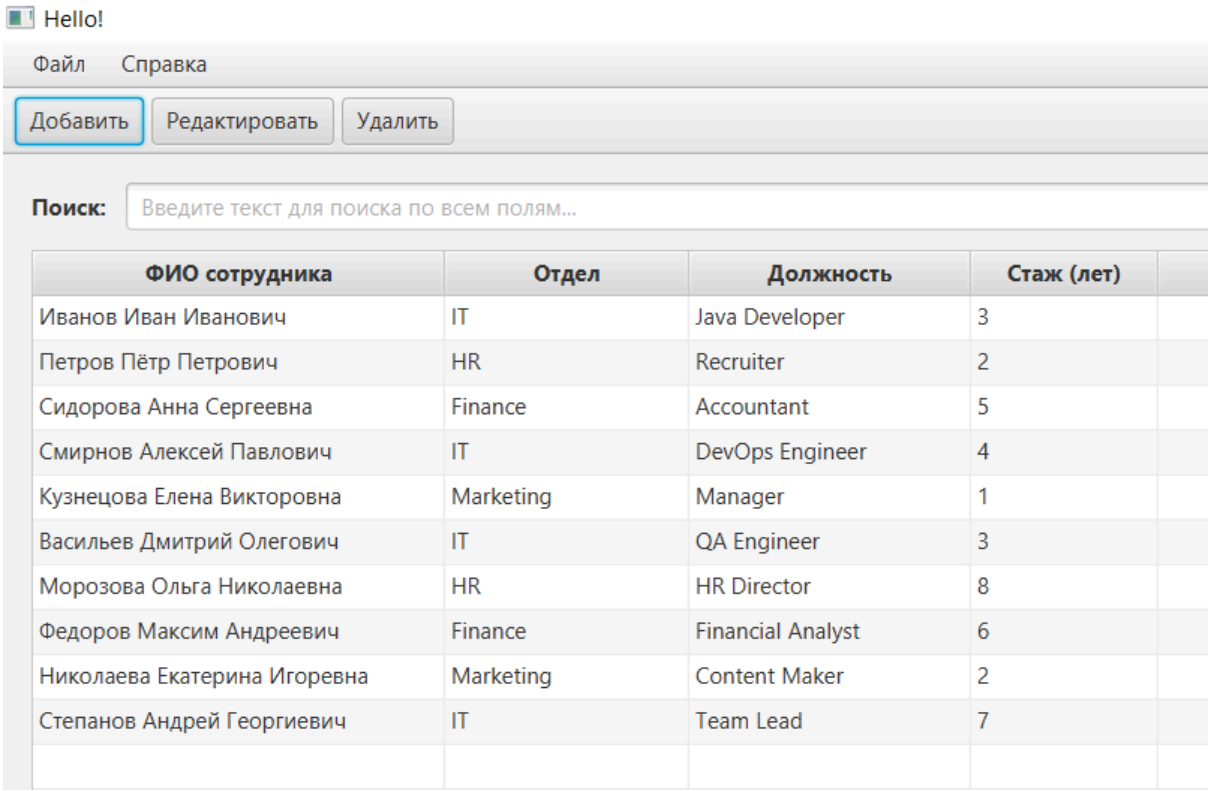
4. РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА (FXML / SCENE BUILDER)

Графический интерфейс состоит из двух ключевых форм:

4.1. Главное окно (hello-view.fxml)

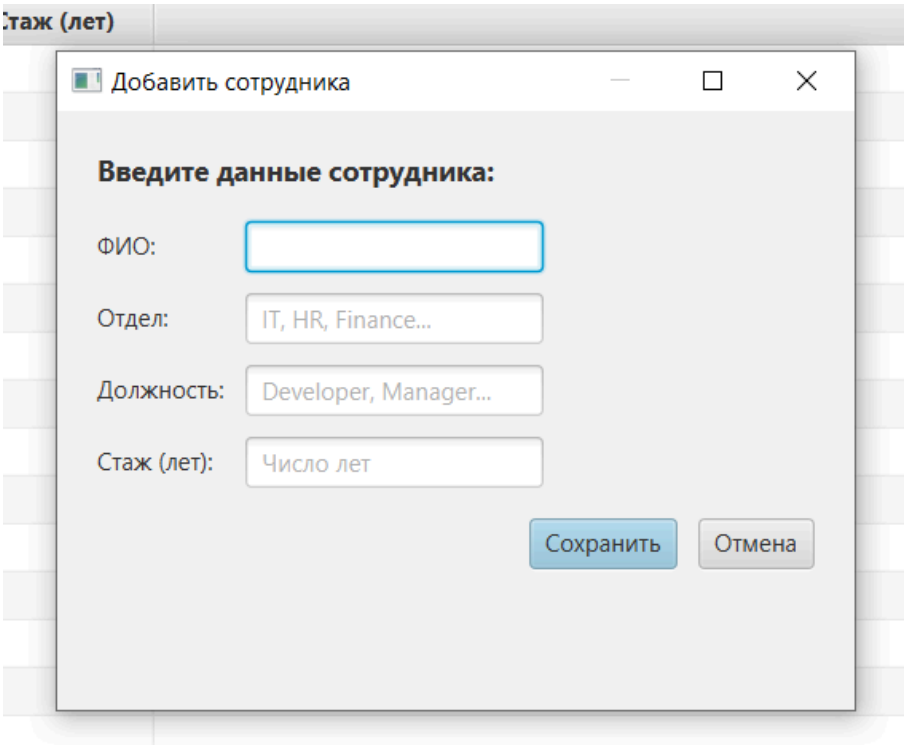
Содержит следующие элементы:

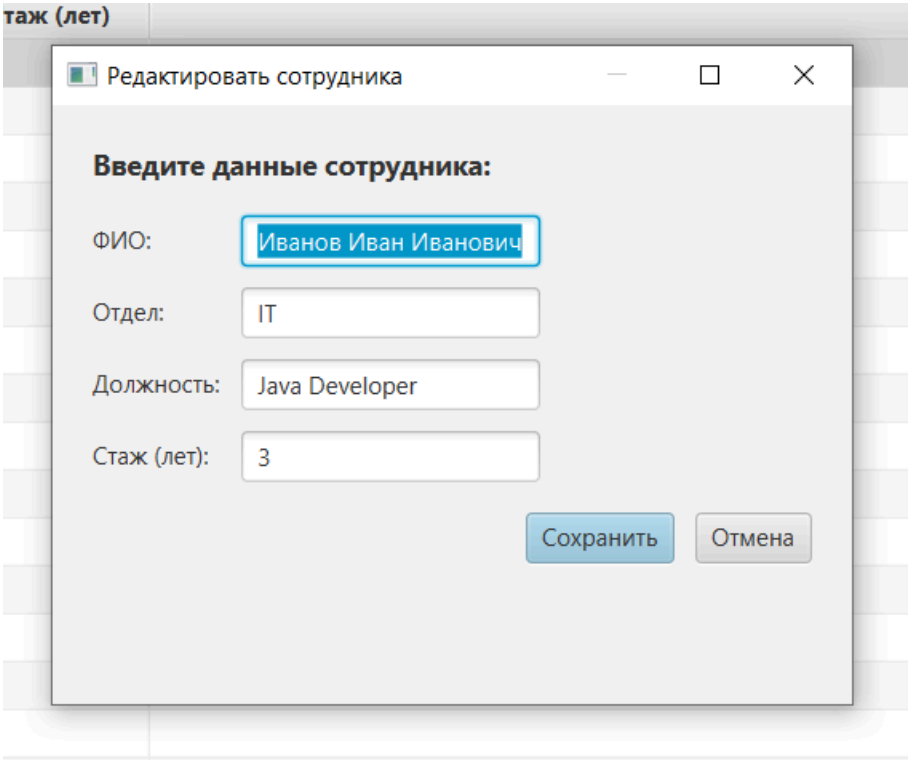
1. **MenuBar (Верхнее меню):** Пункты «Файл» -> «Добавить сотрудника», «Выход», «Справка» -> «О программе».
2. **ToolBar (Панель инструментов):** Кнопки быстрого доступа «Добавить», «Редактировать», «Удалить».
3. **Панель поиска (HBox + TextField):** Текстовое поле для поиска в режиме реального времени.
4. **Центральная таблица (TableView):** Четыре колонки (TableColumn) с привязкой свойств объекта Employee:
    - ФИО сотрудника (fullName)
    - Отдел (department)
    - Должность (position)
    - Стаж в годах (experience)
5. **Строка состояния (StatusBar / Label):** Выводит актуальное количество отображаемых элементов и статус операций.



4.2. Диалоговое окно (employee-dialog.fxml)

Представляет собой модальное окно на базе контейнера VBox и сетки GridPane для выравнивания меток Label и текстовых полей TextField. В нижней части расположены управляющие кнопки «Сохранить» и «Отмена».





5. ЛОГИКА ПРИЛОЖЕНИЯ И ОБРАБОТКА СОБЫТИЙ

5.1. Модель данных Employee.java

```
package com.example.sro1vp17varabdkprv;

public class Employee {
    private String fullName;
    private String department;
    private String position;
    private int experience;

    public Employee(String fullName, String department, String position, int
experience) {
        this.fullName = fullName;
        this.department = department;
        this.position = position;
        this.experience = experience;
    }

    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
```

```
    public String getPosition() { return position; }
    public void setPosition(String position) { this.position = position; }

    public int getExperience() { return experience; }
    public void setExperience(int experience) { this.experience = experience; }
}
```

5.2. Главный контроллер HelloController.java

Контроллер отвечает за инициализацию таблицы, настройку поиска через FilteredList и вызов диалоговых окон:

```
package com.example.sro1vp17varabdkprv;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.collections.transformation.FilteredList;
import javafx.collections.transformation.SortedList;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;

import java.io.IOException;

public class HelloController {

    @FXML private TableView<Employee> employeeTable;
    @FXML private TableColumn<Employee, String> nameColumn;
    @FXML private TableColumn<Employee, String> deptColumn;
    @FXML private TableColumn<Employee, String> posColumn;
    @FXML private TableColumn<Employee, Integer> expColumn;

    @FXML private TextField searchField;
    @FXML private Label statusLabel;

    private ObservableList<Employee> employeeList;

    @FXML
    public void initialize() {
```

```
        nameColumn.setCellValueFactory(new PropertyValueFactory<>("fullName"));
        deptColumn.setCellValueFactory(new PropertyValueFactory<>("department"));
        posColumn.setCellValueFactory(new PropertyValueFactory<>("position"));
        expColumn.setCellValueFactory(new PropertyValueFactory<>("experience"));

        employeeList = FXCollections.observableArrayList(
            new Employee("Иванов Иван Иванович", "IT", "Java Developer", 3),
            new Employee("Петров Пётр Петрович", "HR", "Recruiter", 2),
            new Employee("Сидорова Анна Сергеевна", "Finance", "Accountant", 5),
            new Employee("Смирнов Алексей Павлович", "IT", "DevOps Engineer", 4),
            new Employee("Кузнецова Елена Викторовна", "Marketing", "Manager", 1),
            new Employee("Васильев Дмитрий Олегович", "IT", "QA Engineer", 3),
            new Employee("Морозова Ольга Николаевна", "HR", "HR Director", 8),
            new Employee("Федоров Максим Андреевич", "Finance", "Financial
Analyst", 6),
            new Employee("Николаева Екатерина Игоревна", "Marketing", "Content
Maker", 2),
            new Employee("Степанов Андрей Георгиевич", "IT", "Team Lead", 7)
        );

        FilteredList<Employee> filteredData = new FilteredList<>(employeeList, b ->
true);

        searchField.textProperty().addListener((observable, oldValue, newValue) -> {
            filteredData.setPredicate(employee -> {
                if (newValue == null || newValue.isEmpty()) return true;
                String lowerCaseFilter = newValue.toLowerCase();

                if (employee.getFullName().toLowerCase().contains(lowerCaseFilter))
return true;
                if (employee.getDepartment().toLowerCase().contains(lowerCaseFilter))
return true;
                if (employee.getPosition().toLowerCase().contains(lowerCaseFilter)) return
true;
                if (String.valueOf(employee.getExperience()).contains(lowerCaseFilter))
return true;
                return false;
            });
            statusLabel.setText("Статус: Фильтр применен. Найдено записей: " +
filteredData.size());
        });

        SortedList<Employee> sortedData = new SortedList<>(filteredData);
        sortedData.comparatorProperty().bind(employeeTable.comparatorProperty());
```

```
        employeeTable.setItems(sortedData);
    }

    @FXML
    private void handleAddEmployee() {
        Employee tempEmployee = new Employee("", "", "", 0);
        boolean savedClicked = showEmployeeDialog(tempEmployee, "Добавить
сотрудника");
        if (savedClicked) {
            employeeList.add(tempEmployee);
            statusLabel.setText("Статус: Сотрудник добавлен. Всего записей: " +
employeeList.size());
        }
    }

    @FXML
    private void handleEditEmployee() {
        Employee selectedEmployee =
employeeTable.getSelectionModel().getSelectedItem();
        if (selectedEmployee != null) {
            boolean savedClicked = showEmployeeDialog(selectedEmployee,
"Редактировать сотрудника");
            if (savedClicked) {
                employeeTable.refresh();
                statusLabel.setText("Статус: Данные сотрудника обновлены.");
            }
        } else {
            Alert alert = new Alert(Alert.AlertType.WARNING);
            alert.setTitle("Внимание");
            alert.setHeaderText("Элемент не выбран");
            alert.setContentText("Пожалуйста, выберите сотрудника в таблице для
редактирования.");
            alert.showAndWait();
        }
    }

    private boolean showEmployeeDialog(Employee employee, String title) {
        try {
            FXMLLoader loader = new
FXMLLoader(HelloApplication.class.getResource("employee-dialog.fxml"));
            VBox page = loader.load();

            Stage dialogStage = new Stage();
            dialogStage.setTitle(title);
```

```
            dialogStage.initModality(Modality.WINDOW_MODAL);
            dialogStage.initOwner(employeeTable.getScene().getWindow());
            Scene scene = new Scene(page);
            dialogStage.setScene(scene);

            EmployeeDialogController controller = loader.getController();
            controller.setDialogStage(dialogStage);
            controller.setEmployee(employee);

            dialogStage.showAndWait();
            return controller.isSavedClicked();
        } catch (IOException e) {
            e.printStackTrace();
            return false;
        }
    }

    @FXML
    private void handleDeleteEmployee() {
        Employee selected = employeeTable.getSelectionModel().getSelectedItem();
        if (selected != null) {
            employeeList.remove(selected);
            statusLabel.setText("Статус: Сотрудник удален. Всего записей: " +
employeeList.size());
        } else {
            Alert alert = new Alert(Alert.AlertType.WARNING);
            alert.setTitle("Внимание");
            alert.setHeaderText("Элемент не выбран");
            alert.setContentText("Пожалуйста, выберите сотрудника в таблице для
удаления.");
            alert.showAndWait();
        }
    }

    @FXML private void handleExit() { System.exit(0); }

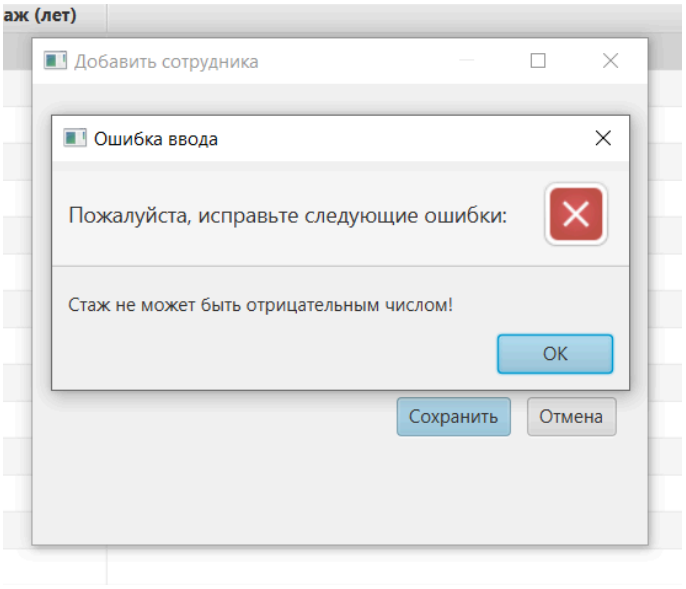
    @FXML
    private void handleAbout() {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle("О программе");
        alert.setHeaderText("СРО №1. Визуальное программирование");
        alert.setContentText("Вариант 17: Employee Management
System\nРазработано для выполнения СРО.");
        alert.showAndWait();
```

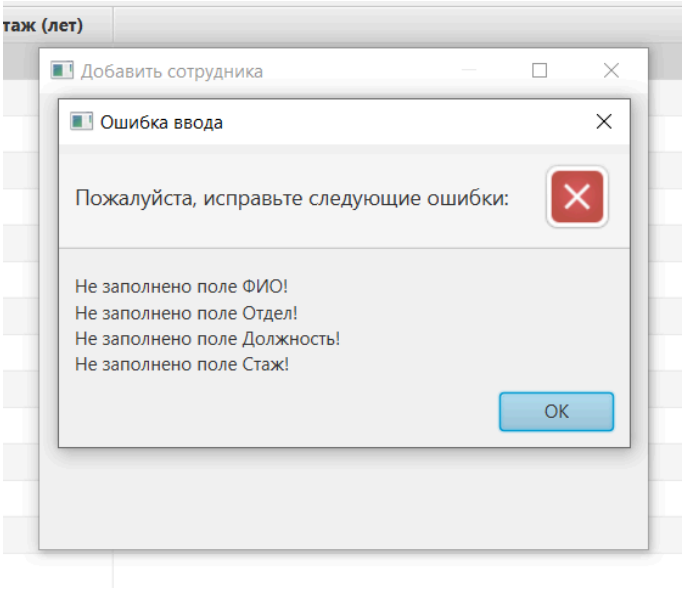
```
    }
}
```

6. ВАЛИДАЦИЯ ДАННЫХ И ОБРАБОТКА ОШИБОК

Контроллер диалогового окна EmployeeDialogController.java выполняет проверку корректности введённых данных перед сохранением объекта:

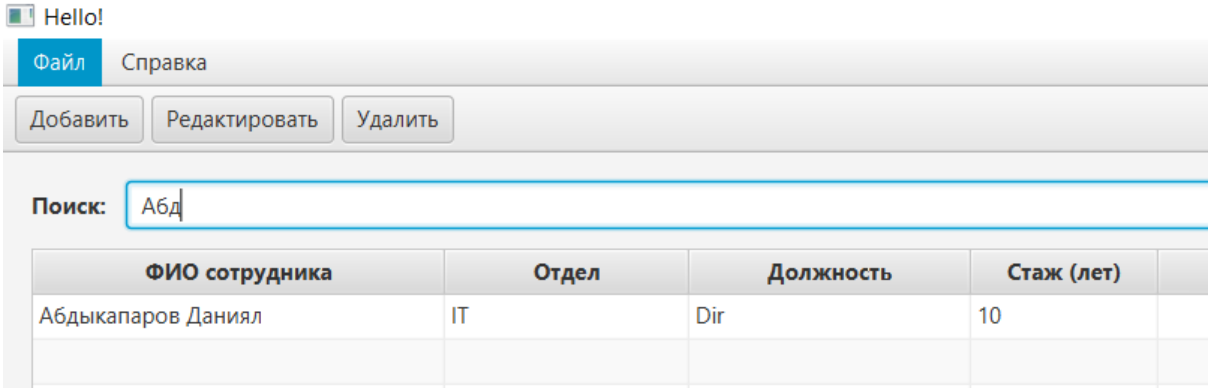
1. Проверка текстовых полей на пустоту.
2. Проверка числового значения поля «Стаж» (преобразование через Integer.parseInt).
3. Контроль отрицательных чисел.
4. Вывод критических сообщений в диалоговом окне типа Alert.AlertType.ERROR без падения программы.





7. ТЕСТИРОВАНИЕ И ПОЛЬЗОВАТЕЛЬСКИЕ СЦЕНАРИИ

№	Тест-кейс	Ожидаемый результат	Статус
1	Запуск приложения	Отображается таблица с 10 предустановленными сотрудниками.	Успешно
2	Поиск "IT" в поле ввода	В таблице остаются только сотрудники отдела IT.	Успешно
3	Добавление корректной записи	Новый сотрудник отображается внизу таблицы.	Успешно
4	Ввод букв в поле "Стаж"	Выводится окно Alert с сообщением об ошибке.	Успешно
5	Нажатие "Удалить" без выбора строки	Выводится сообщение Alert.WARNING.	Успешно
6	Нажатие "Редактировать"	Поля формы заполняются выбранным сотрудником, изменения сохраняются.	Успешно



8. ИНСТРУКЦИЯ ПО ЗАПУСКУ И РАЗВЕРТЫВАНИЮ

Требования к окружению:

- JDK 17 или JDK 21
- Apache Maven 3.8+

Команды для сборки и запуска:

```
# Клонирование репозитория
git clone https://github.com/Admatist/PHP.git

# Сборка проекта
mvn clean package

# Запуск приложения через Maven плагин JavaFX
mvn javafx:run
```

9. ЗАКЛЮЧЕНИЕ

В ходе выполнения СРО №1 по дисциплине «Визуальное программирование» было успешно разработано графическое приложение **«Employee Management System»**.

Были выполнены все поставленные задачи:

- Изучены и применены на практике принципы связывания FXML-разметки и Java-контроллеров.
- Построена многооконная архитектура приложения с модальными окнами.
- Реализованы CRUD-операции (создание, чтение, обновление, удаление) над коллекцией объектов.
- Настроена фильтрация данных в реальном времени и валидация пользовательского ввода.
- Исходный код размещён в репозитории GitHub.